

Memory Controllers

Introduction

Memory controllers are essential components that manage communication between processors and memory devices. They exist throughout the memory hierarchy - from cache controllers to DRAM controllers to SSD controllers. As we move to longer-latency memories (DRAM, SSDs, emerging technologies), controller design becomes significantly more complex.

Key Insight: Long-latency memories are harder to control because they:

- Are typically off-chip (requiring external interfaces)
- Have many timing constraints to manage
- Need sophisticated scheduling algorithms
- Must handle multiple concurrent requests

Why Memory Controllers Are Critical

Memory controllers have become increasingly important and complex for several reasons:

Growing Complexity Factors

- **Multiple memory types:** DDR, LPDDR, GDDR, HBM, 3D-stacked DRAM
- **Multiple requestors:** Multi-core processors, GPUs, accelerators
- **Competing goals:** Performance, fairness, QoS, energy efficiency, security
- **Technology constraints:** Refresh requirements, timing parameters, power modes

Core Responsibilities

1. **Ensure correct operation** - obey all timing and refresh requirements
2. **Translate requests to commands** - convert addresses to bank/row/column operations
3. **Buffer and schedule requests** - optimize for performance and fairness
4. **Manage power** - switch between power states intelligently
5. **Handle errors** - implement ECC and other reliability mechanisms

Types of Memory and Controllers

DRAM Types

Different DRAM types have different interfaces optimized for specific purposes:

- **DDR (DDR3, DDR4, DDR5):** Commodity interfaces for general computing

- **LPDDR:** Low-power interfaces for mobile devices
- **GDDR:** Graphics-optimized with high bandwidth
- **HBM:** 3D-stacked high-bandwidth memory
- **Specialized:** Low-latency DRAM (expensive, high-cost)

Important: The underlying DRAM microarchitecture is fundamentally the same across types. What differs is:

- Interface protocols
- Timing parameters
- Number of banks
- Pin configurations
- Analog signaling (the most expensive part)

Challenge with Flexible Controllers

Ideally, one controller could support multiple DRAM types, but this is difficult because:

- Each type has different analog interfaces
- High-speed analog interfaces are expensive to design
- Different pin configurations required
- Supporting multiple types significantly increases cost and complexity

DRAM Controllers

DRAM Structure Hierarchy

Understanding DRAM structure is key to controller design:

```
Processor → Memory Controller → Channels → Ranks → Chips → Banks → Subarrays → Rows/Columns
```

- **Subarrays:** Basic storage units within banks
- **Banks:** Independent units that can operate in parallel
- **Ranks:** Multiple chips sharing the same command/address bus
- **Channels:** Independent data paths to memory

Row Buffer Management

DRAM uses a row buffer (sense amplifiers) to access data:

1. **Activate:** Opens a row into the row buffer
2. **Read/Write:** Access data from the open row
3. **Precharge:** Closes the row, prepares bank for next access

Key Concepts:

- **Row hit:** Next access to same row (fast, ~13-15ns)
- **Row conflict:** Next access to different row (slow, requires precharge + activate, ~50ns)
- **Row empty:** No row currently open

Row Buffer Policies

Open-Row Policy:

- Keep row open after access
- Pros: Fast if next access is to same row (row hit)
- Cons: Wastes energy, slower if next access is to different row (conflict)

Closed-Row Policy:

- Close row immediately after access (if no pending requests need it)
- Pros: Avoids row conflicts, saves energy
- Cons: Extra activate if next access is to same row

Adaptive Policy (Best):

- Predict whether next access will be to same row
- Choose open/closed based on prediction
- Used in modern controllers

Command Types

- **Activate (ACT):** Open a row
- **Precharge (PRE):** Close a row
- **Read (RD):** Read from open row
- **Write (WR):** Write to open row
- **Refresh (REF):** Restore charge in cells

Scheduling Policies

Memory controllers must decide which request to service next. This is called **scheduling** or **arbitration**.

Simple Policies

FCFS (First-Come-First-Served):

- Process requests in arrival order
- Simple but poor performance

- No consideration of memory state

Random:

- Select request randomly
- Even simpler, rarely used

Row-Buffer-Aware Policies

FR-FCFS (First-Ready FCFS):

- Prioritize row-buffer hits over misses
- Among same type, prioritize older requests
- Goal: Maximize row-buffer hit rate and throughput

Command-Level FR-FCFS:

- Convert requests to commands (ACT, PRE, RD, WR)
- Prioritize column commands (RD, WR) over row commands (ACT, PRE)
- Within each group, prioritize older commands

Read vs Write Scheduling

Challenge: DRAM bus direction must switch between reads and writes

- Read-to-write delay: ~11-15ns
- Write-to-read delay: ~7.5ns
- Switching has overhead

Strategy:

- Batch reads together, batch writes together
- Writes are usually less critical (writebacks from cache)
- But must service writes eventually or buffers fill up
- Sophisticated controllers minimize mode transitions

Advanced Scheduling Factors

Controllers can consider many factors for prioritization:

- **Request age:** Older vs newer
- **Row-buffer status:** Hit vs miss vs empty
- **Request type:** Read vs write vs prefetch
- **Criticality:** Load miss vs store miss
- **Thread priority:** Which core/thread is waiting
- **Interference:** Impact on other cores

- **Memory-level parallelism:** Can latency be hidden?

Challenge: More factors = more complex priority logic = harder to build

DRAM Timing Constraints

DRAM has many timing constraints (100+ parameters) that controllers must obey for correct operation.

Why Multiple Timing Parameters?

A single worst-case timing (e.g., 50ns for everything) would severely limit performance. Different operations have different physical requirements, so standards define specific constraints for each transition.

Key Timing Parameters

Constraint	Description	Example Value
tRCD	ACT to RD/WR delay	13-15ns
tRAS	ACT to PRE delay	35ns
tRP	PRE to ACT delay	13-15ns
tRC	ACT to ACT (same bank)	50ns
tCAS	Column access latency	13-15ns
tRTW	Read to write delay	11-15ns
tWTR	Write to read delay	7.5ns
tREFI	Refresh interval	7.8μs

Scope of Constraints

- **Bank-level:** tRCD, tRAS, tRP, tRC
- **Channel/Bus-level:** tRTW, tWTR
- **Rank-level:** Some refresh constraints

Physical Reasons

- **tRCD:** Time for sense amplifiers to stabilize after activate
- **tRAS:** Time for full row restoration before precharge
- **tRTW/tWTR:** Time to switch bus direction
- **tREFI:** Maximum time between refreshes before data loss

Reducing Latency

Recent research shows significant guard bands in these parameters:

- Can reduce latency by profiling individual chips
- Account for temperature, voltage, manufacturing variation
- But must maintain some minimum for correct operation